

Name - Saikat Sheet
Batch - 25SUB5936_Linux System Programming
ID - 68500

Task 12:

TCP Socket Programming in C

- Develop a simple client-server application using TCP sockets in C:
- The server listens on a specified port and accepts a client connection.
- Once connected, the server sends a message "Welcome to the Server!" to the client.
- The client receives the message and prints it to the console.

Code:

Server.c

```
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

#define PORT 8080
int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char *message = "Welcome to the Server!";

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
```

```

    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Set socket options to reuse address
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// Bind socket
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen
if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d\n", PORT);

// Accept connection
new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t *)&addrlen);
if (new_socket < 0) {
    perror("accept failed");
    exit(EXIT_FAILURE);
}

// Send message
send(new_socket, message, strlen(message), 0);

```

```
printf("Message sent to client\n");

close(new_socket);
close(server_fd);
return 0;
}
```

client.c

```
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>

#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};

    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket creation failed");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        perror("Invalid address");
    }
}
```

```

        return -1;
    }

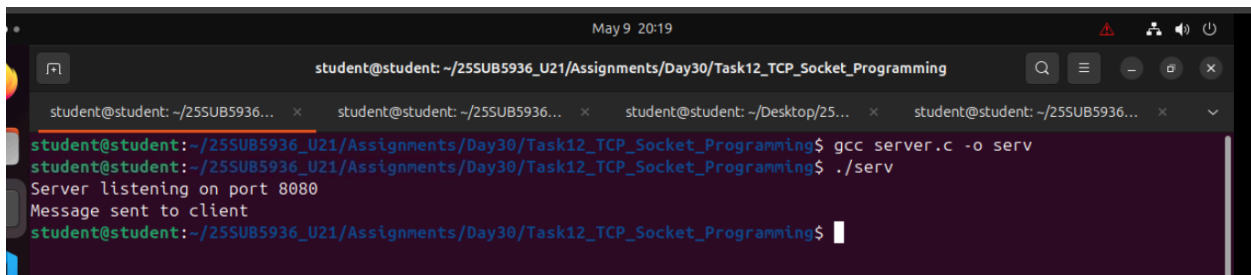
    // Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("connection failed");
        return -1;
    }

    // Receive message
    read(sock, buffer, BUFFER_SIZE);
    printf("Server: %s\n", buffer);

    close(sock);
    return 0;
}

```

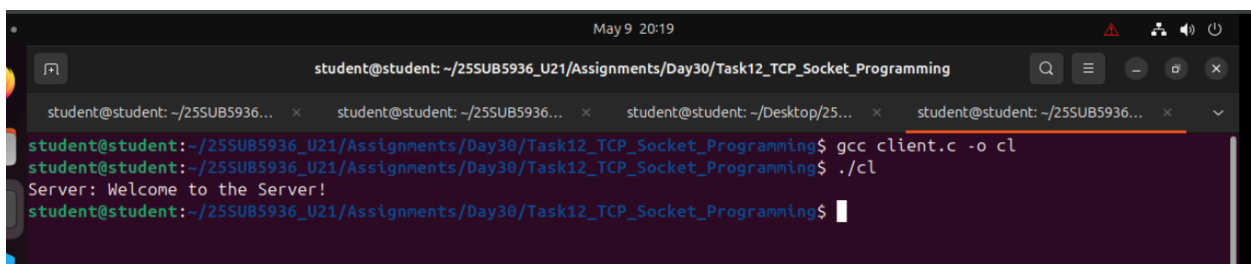
Output



```

student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming
student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming$ gcc server.c -o serv
student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming$ ./serv
Server listening on port 8080
Message sent to client
student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming$

```



```

student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming
student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming$ gcc client.c -o cl
student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming$ ./cl
Server: Welcome to the Server!
student@student: ~/25SUB5936_U21/Assignments/Day30/Task12_TCP_Socket_Programming$

```